

CONTENTS

Intratabilidad y NP-Complejidad	
Guía de estudio con tolerancia cero — Módulo 8, Programación 3 (FING UdelAR)	
0. Por qué existe este tema	
1. Las clases P, NP, NP-Hard y NP-Completo	
1.1 Clase P	
1.2 Clase NP	
1.3 Por qué $P \subseteq NP$ (y por qué $P=NP$ sigue abierto)	
1.4 NP-Hard y NP-Completo	
2. Reducciones polinomiales: qué son y qué demuestran	
2.1 Qué demuestra exactamente $A \leq_p B$	
2.2 El error más común: invertir la dirección de la reducción	
3. Método paso a paso para demostrar que un problema X es NP-Completo	
Paso (i) — Demostrar que $X \in NP$	
Paso (ii) — Elegir un problema Y ya conocido como NP-Completo	
Paso (iii) — Construir la reducción $Y \leq_p X$	
Paso (iv) — Demostrar la equivalencia en ambas direcciones	
4. Ejemplo completo y detallado: $3\text{-SAT} \leq_p \text{Independent Set}$	
Paso (i) — $\text{Independent Set} \in NP$	
Paso (ii) — Problema conocido: 3-SAT	

Paso (iii) — Construcción de la reducción

Paso (iv) — Equivalencia en ambas direcciones

Verificación con una instancia numérica concreta

5. Problemas NP-Completos de referencia

6. Trampas típicas de examen

Ejercicios de repaso rápido (con respuesta)

Resumen ultra-rápido (2 minutos)

Intratabilidad y NP-Complejidad

GUÍA DE ESTUDIO CON TOLERANCIA CERO — MÓDULO 8, PROGRAMACIÓN 3 (FING UDELAR)

0. Por qué existe este tema

Hasta el capítulo 7 del curso (algoritmos ávidos, divide y vencerás, programación dinámica, flujos), cada vez que aparecía un problema nuevo la pregunta era “¿cuál técnica lo resuelve en tiempo polinomial?”. Este módulo cambia la pregunta de raíz: **¿existe siquiera un algoritmo polinomial para este problema, o no?**

La mayoría de los problemas de optimización combinatoria interesantes (rutas, empaquetamiento, satisfacción de restricciones, selección de subconjuntos) tienen una estructura combinatoria brutal: el espacio de soluciones candidatas crece exponencialmente con el tamaño de la entrada. Para algunos de estos problemas (camino más corto, flujo máximo, MST) encontramos atajos algorítmicos que evitan la explosión exponencial. Para muchísimos otros — que se **ven** casi idénticos a los anteriores en su enunciado — nadie ha encontrado jamás un atajo así, a pesar de décadas de intentos. Vertex Cover y Shortest Path se parecen en el papel; en la práctica, uno se resuelve en $O(n)$ y el otro no tiene ningún algoritmo conocido mejor que fuerza bruta esencialmente disfrazada.

La teoría de NP-complejidad no resuelve esa asimetría (nadie sabe si $P=NP$), pero da una herramienta muy poderosa y **totalmente al alcance de un examen**: en vez de demostrar “este problema es difícil” desde cero (lo cual requeriría resolver el problema abierto más famoso de la computación), alcanza con demostrar que es **al menos tan difícil como otro problema que ya sabemos que es difícil**, mediante una construcción explícita — una **reducción**. Ese es el 90% del contenido evaluable de este módulo: saber construir y verificar una reducción, no debatir filosóficamente si $P=NP$.

Regla de oro de todo el módulo: **nunca** se pide (ni se puede pedir, porque es un problema abierto) demostrar $P \neq NP$. Lo que SIEMPRE se pide es: dado un problema X nuevo, demostrar que es NP-Completo *relativo* a problemas ya conocidos, mediante una reducción explícita y verificable en ambas direcciones.

1. Las clases P, NP, NP-Hard y NP-Completo

1.1 CLASE P

Definición. Un problema de decisión (respuesta sí/no) pertenece a la clase **P** si existe un algoritmo que lo resuelve en tiempo **polinomial** en el tamaño de la entrada, es decir, en tiempo $O(n^c)$ para alguna constante c . Informalmente: $P =$ “problemas que se pueden **resolver** eficientemente”.

Ejemplos ya vistos en el curso: Ordenamiento, Camino Más Corto (Dijkstra/Bellman-Ford), Flujo Máximo, MST, 2-coloreo/bipartición (Práctico 3, Ej. 7), la mayoría de los problemas resueltos con programación dinámica de este curso.

1.2 CLASE NP

Definición formal (certificación eficiente). Un problema de decisión pertenece a la clase **NP** (*Nondeterministic Polynomial*) si, para toda instancia cuya respuesta es “sí”, existe un **certificado** (también llamado *testigo* o *solución propuesta*) de tamaño polinomial en el tamaño de la entrada, tal que un **algoritmo verificador** puede comprobar en tiempo polinomial que ese certificado efectivamente prueba que la respuesta es “sí”.

Informalmente: $NP =$ “problemas donde, **si alguien me da la solución**, puedo chequear rápido que es correcta” — aunque encontrarla desde cero pueda tardar exponencial.

NP no significa “No Polinomial”. Es un error de lectura frecuentísimo. NP significa “Polinomial No-determinista” — la clase de problemas **verificables** en tiempo polinomial, no la clase de problemas que no se pueden resolver en tiempo polinomial. De hecho, todo problema de P también está en NP (ver 1.3).

Ejemplo 1.1 — Certificado y verificador para Independent Set.

Instancia: grafo $G=(V,E)$, entero k . Pregunta: ¿existe $S \subseteq V$, $|S| \geq k$, tal que ningún par de nodos de S es adyacente?

Certificado: el propio conjunto S (una lista de a lo sumo $|V|$ nodos).

Verificador: (1) chequear $|S| \geq k$ — $O(1)$ si S viene con su tamaño, $O(k)$ si hay que contar.
 (2) Para cada par de nodos en S , chequear que no exista la arista entre ellos — $O(k^2)$ pares, cada chequeo de adyacencia $O(1)$ con matriz de adyacencia o $O(\deg)$ con lista.
 Total: $O(k^2)$ o $O(V^2)$, polinomial en el tamaño de G .

Conclusión: Independent Set $\in NP$.

Ejemplo 1.2 — Certificado y verificador para SAT.

Instancia: conjunto de cláusulas $C_1, \dots, C_k$ sobre variables $x_1, \dots, x_n$. **Pregunta:** ¿existe una asignación de verdad que satisface todas las cláusulas?

Certificado: la asignación de verdad propuesta (n bits, uno por variable).

Verificador: para cada cláusula, recorrer sus términos y chequear que al menos uno se evalúe a verdadero con la asignación dada — $O(\text{tamaño total de las cláusulas})$, lineal en la entrada.

Conclusión: $\text{SAT} \in \text{NP}$.

1.3 POR QUÉ $P \subseteq \text{NP}$ (Y POR QUÉ $P=\text{NP}$ SIGUE ABIERTO)

$P \subseteq \text{NP}$ siempre se cumple, y es fácil de ver: si un problema está en P , existe un algoritmo A que lo resuelve completo (no solo verifica) en tiempo polinomial. Entonces puedo usar A mismo como “verificador” que ignora el certificado propuesto y resuelve la instancia desde cero — si A dice “sí”, listo, no hace falta ni mirar el certificado. Esto demuestra que todo problema resoluble en tiempo polinomial también es verificable en tiempo polinomial.

La pregunta **$P = \text{NP}$?** (¿la contención es en realidad una igualdad?) es el problema abierto más famoso de las ciencias de la computación (uno de los siete “Problemas del Milenio” del Clay Institute, con premio de US\$1.000.000). Nadie ha logrado demostrar ni que $P=\text{NP}$ ni que $P \neq \text{NP}$. La sospecha mayoritaria de la comunidad es que **$P \neq \text{NP}$** (que hay problemas verificables rápido pero no resolubles rápido), sustentada en que miles de problemas de NP han resistido décadas de intentos de encontrarles un algoritmo polinomial — pero eso es evidencia circunstancial, no una demostración.

Lo que **SÍ** se puede demostrar (y es la columna vertebral de este módulo) es una afirmación condicional mucho más débil pero perfectamente rigurosa: **“si alguna vez alguien encuentra un algoritmo polinomial para este problema NP-Completo específico, entonces automáticamente existe un algoritmo polinomial para todos los problemas de NP”** (es decir, $P=\text{NP}$). Como miles de personas llevan décadas buscando ese algoritmo sin éxito para docenas de problemas NP-Completo distintos, la conjetura de que $P \neq \text{NP}$ se sostiene — pero repetimos: es una conjetura, no una demostración, y el examen NUNCA va a pedir “demostrar” $P \neq \text{NP}$.

1.4 NP-HARD Y NP-COMPLETO

NP-Hard (NP-Difícil). Un problema X es NP-Hard si **todo** problema $Y \in \text{NP}$ se puede reducir polinomialmente a X ($Y \leq_p X$ para todo $Y \in \text{NP}$). Informalmente: X es “al menos tan difícil como el problema más difícil de NP”. **Un problema NP-Hard NO tiene por qué estar en NP** —

puede ser incluso indecidible, o un problema de optimización (no de decisión) que ni siquiera tiene la forma sí/no necesaria para pertenecer a NP.

NP-Completo. Un problema X es NP-Completo si cumple **ambas** condiciones a la vez:

1. $X \in \text{NP}$ (es verificable en tiempo polinomial).
2. X es NP-Hard (todo problema de NP se reduce a X).

Informalmente: los problemas NP-Completo son los **más difíciles dentro de NP** — si alguno de ellos se resolviera en tiempo polinomial, **todos** los problemas de NP se resolverían en tiempo polinomial (porque todos se reducen a él), es decir, se demostraría $P=NP$.

Clase	¿Está en NP?	¿Es “al menos tan difícil” como todo NP?
P	Sí (ver 1.3)	No necesariamente
NP	Sí (por definición)	No necesariamente
NP-Hard	No necesariamente	Sí, siempre
NP-Completo	Sí	Sí

Error frecuente de tolerancia cero: usar “NP-Hard” y “NP-Completo” como sinónimos. NP-Hard es una condición **necesaria pero no suficiente** para NP-Completo — falta también estar en NP. Ejemplo típico de problema NP-Hard que no es NP-Completo: la versión de **optimización** de un problema (ej. “¿cuál es el tamaño del Vertex Cover mínimo?”, en vez de “¿existe un Vertex Cover de tamaño $\leq k$?”). La versión de optimización no tiene una respuesta sí/no verificable en tiempo polinomial de forma directa (verificar que un número es efectivamente el mínimo requeriría, en principio, descartar todos los tamaños menores), así que es NP-Hard pero no se ajusta a la definición de NP tal cual está dada. Por eso en TODO este módulo trabajamos con la **versión de decisión** de cada problema (con un parámetro k y una pregunta sí/no), nunca directamente con la versión de optimización.

2. Reducciones polinomiales: qué son y qué demuestran

Definición. Un problema A se reduce polinomialmente a un problema B (notación $A \leq_p B$) si existe una función f, computable en tiempo polinomial, que transforma toda instancia I_A de A en una instancia $f(I_A)$ de B, tal que:

I_A es una instancia “sí” de $A \Leftrightarrow f(I_A)$ es una instancia “sí” de B .

Es decir, la transformación preserva la respuesta en **ambas direcciones** (es una equivalencia lógica, no una implicación en un solo sentido).

2.1 QUÉ DEMUESTRA EXACTAMENTE $A \leq_p B$

La consecuencia directa de $A \leq_p B$ es:

Si $B \in P$, entonces $A \in P$.

Intuición: si tengo un algoritmo polinomial para B , puedo resolver A así: tomo la instancia de A , la transformo a una instancia de B con f (tiempo polinomial), la resuelvo con el algoritmo de B (tiempo polinomial), y devuelvo la misma respuesta. La composición de dos pasos polinomiales es polinomial, así que A queda resuelto en tiempo polinomial total.

El contrapositivo es la herramienta real que se usa en el 100% de los ejercicios de este módulo:

Si A es NP-Hard (o ya se sabe que $A \notin P$, asumiendo $P \neq NP$) y $A \leq_p B$, entonces B también es NP-Hard.

Esto es exactamente el contrapositivo lógico de la implicación de arriba: si B estuviera en P , por la implicación A también estaría en P — pero sabemos que A no lo está (o es NP-Hard), contradicción. Por lo tanto B no puede estar en P (bajo la suposición estándar $P \neq NP$), y de hecho hereda la “dificultad” completa de A .

Traducción práctica para el examen: para demostrar que un problema NUEVO X es NP-Hard, hay que **encontrar un problema Y que YA se sabe NP-Completo, y construir la reducción $Y \leq_p X$** (de Y hacia X). Nunca al revés. Ver la trampa típica en la sección 6.

2.2 EL ERROR MÁS COMÚN: INVERTIR LA DIRECCIÓN DE LA REDUCCIÓN

Error de tolerancia cero — la trampa #1 de todo el módulo: para demostrar que X es NP-Hard, hay que reducir un problema **ya conocido** Y (NP-Completo) **hacia** X ($Y \leq_p X$), es decir, transformar instancias de Y en instancias de X . Reducir **al revés** ($X \leq_p Y$, transformar instancias de X en instancias de Y) demuestra otra cosa completamente distinta — como mucho, que X no es “más difícil” que Y (si Y estuviera en P , X también lo estaría), que es información inútil cuando lo que se busca es una cota inferior de dificultad para X .

Regla mnemotécnica: "**reduzco DESDE lo difícil conocido HACIA lo nuevo que quiero clasificar**". Si dudás sobre la dirección, preguntate: "¿mi transformación resuelve instancias del problema difícil-conocido usando al problema nuevo como caja negra?" — si la respuesta es sí, la dirección es correcta.

3. Método paso a paso para demostrar que un problema X es NP-Completo

Esto es lo más importante del módulo — el "cómo" que hay que poder ejecutar sin errores en el examen. Son **cuatro pasos obligatorios**, y omitir cualquiera de ellos (incluso si el resultado final "se ve bien") se considera demostración incompleta.

PASO (I) — DEMOSTRAR QUE $X \in NP$

Dar explícitamente:

- El **certificado**: qué forma tiene una solución propuesta (normalmente: el subconjunto, la asignación, la selección de elementos que testifica la respuesta "sí").
- El **verificador**: el algoritmo que, dado el certificado, chequea en tiempo polinomial que efectivamente cumple las condiciones pedidas por el problema.
- El **análisis de complejidad** del verificador, mostrando explícitamente que es polinomial en el tamaño de la entrada original.

PASO (II) — ELEGIR UN PROBLEMA Y YA CONOCIDO COMO NP-COMPLETO

No se puede reducir "desde cero" — hace falta apoyarse en un problema de la lista de referencia (sección 5). La elección de Y no es arbitraria: conviene elegir el problema conocido **cuya estructura combinatoria se parezca más** a la de X (mismo tipo de restricciones: selección de subconjuntos, satisfacción de cláusulas, cobertura, etc.). Elegir mal Y no invalida la demostración si igual se logra construir la reducción, pero puede hacerla mucho más difícil de escribir — algunas reducciones son triviales desde un Y y muy retorcidas desde otro.

PASO (III) — CONSTRUIR LA REDUCCIÓN $Y \leq_P X$

Definir explícitamente la función f que transforma **toda** instancia I_Y de Y en una instancia $f(I_Y)$ de X:

- Especificar la construcción con total precisión (qué se crea, con qué parámetros, en función de los datos de I_Y).
- Justificar que f se puede calcular en **tiempo polinomial** en el tamaño de I_Y (contar cuántos elementos nuevos se crean, y que la construcción de cada uno toma tiempo polinomial).

PASO (IV) — DEMOSTRAR LA EQUIVALENCIA EN AMBAS DIRECCIONES

Esta es la parte que más se recorta en exámenes mal resueltos, y es tolerancia cero: **hay que demostrar las dos implicaciones por separado**, nunca alcanza con mostrar una sola dirección “porque la otra es obvia”.

- $(\Rightarrow)$ “ I_Y es instancia sí de Y ” $\Rightarrow$ “ $f(I_Y)$ es instancia sí de X ”: normalmente se toma la solución (certificado) que hace que I_Y sea “sí”, y se construye explícitamente, a partir de ella, un certificado que demuestra que $f(I_Y)$ es “sí” en X .
- $(\Leftarrow)$ “ $f(I_Y)$ es instancia sí de X ” $\Rightarrow$ “ I_Y es instancia sí de Y ”: se toma un certificado que resuelve $f(I_Y)$ en X , y se demuestra que necesariamente induce (o se puede transformar en) un certificado que resuelve I_Y en Y .

Muchas construcciones son “simétricas” (la misma correspondencia certificado $\leftrightarrow$ certificado funciona en las dos direcciones, como en el ejemplo de la sección 4), lo cual hace tentador escribir una sola prueba y decir “análogamente para la otra dirección”. Aun así, en un examen conviene escribir explícitamente el argumento de **ambas** direcciones, aunque sean cortas — un corrector de tolerancia cero puede restar puntos por una demostración “de una sola vía” incluso si es matemáticamente cierta la otra dirección por simetría.

Plantilla mental de los 4 pasos (memorizar el orden):

- $X \in \text{NP} \rightarrow$ certificado + verificador + complejidad.
- elegir Y NP-Completo conocido, parecido en estructura.
- construir f : instancia de $Y \rightarrow$ instancia de X , en tiempo polinomial.
- demostrar $(\Rightarrow)$ y $(\Leftarrow)$ por separado, explícitamente.

4. Ejemplo completo y detallado: 3-SAT $\leq_p$ Independent Set

Esta es una de las reducciones clásicas del capítulo 8 de Kleinberg & Tardos y sirve como plantilla de estilo para casi todas las reducciones de este práctico.

3-SAT (ya se toma como conocido NP-Completo — es la base de toda la teoría, por el Teorema de Cook-Levin, fuera del alcance de este curso demostrar desde cero, pero SAT/3-SAT es el punto de partida universal): instancia = conjunto de cláusulas $C_1, \dots, C_k$, cada una disyunción de exactamente 3 literales sobre variables $x_1, \dots, x_n$. Pregunta: ¿existe una asignación de verdad que satisfice las k cláusulas simultáneamente?

Independent Set: instancia = grafo $G=(V,E)$, entero m . Pregunta: ¿existe $S \subseteq V$ con $|S| \geq m$ tal que ningún par de nodos de S es adyacente?

PASO (I) — INDEPENDENT SET $\in$ NP

Ya demostrado en el Ejemplo 1.1: certificado = el conjunto S propuesto; verificador chequea $|S| \geq m$ y que ningún par de nodos de S tenga arista entre sí, en $O(m^2)$ o $O(V^2)$ — polinomial. ✓

PASO (II) — PROBLEMA CONOCIDO: 3-SAT

Se usa 3-SAT como problema de partida (dado como conocido NP-Completo).

PASO (III) — CONSTRUCCIÓN DE LA REDUCCIÓN

Dada una instancia de 3-SAT con cláusulas $C_1, \dots, C_k$, se construye un grafo $G=(V,E)$ así:

1. **Nodos**: por cada cláusula C_i (que tiene exactamente 3 literales), crear 3 nodos, uno por cada literal que aparece en esa cláusula. Total: $|V| = 3k$ nodos.
2. **Aristas “de triángulo” (dentro de cada cláusula)**: conectar entre sí los 3 nodos que corresponden a la misma cláusula C_i (forman un triángulo completo — K_3). Esto garantiza que un conjunto independiente puede tomar **a lo sumo un literal de cada cláusula** (porque los 3 nodos de una cláusula están todos conectados entre sí).
3. **Aristas “de conflicto” (entre cláusulas distintas)**: conectar dos nodos de cláusulas *distintas* si y solo si representan literales **opuestos** de la misma variable (uno es x_j y el otro es $\neg x_j$). Esto garantiza que un conjunto independiente nunca puede elegir simultáneamente un literal y su negación.
4. **Parámetro**: $m = k$ (se pregunta por un conjunto independiente de tamaño **igual** a la cantidad de cláusulas).

Tiempo de construcción: k cláusulas $\rightarrow 3k$ nodos ($O(k)$); las aristas de triángulo son 3 por cláusula ($O(k)$ en total); las aristas de conflicto son a lo sumo $O((3k)^2) = O(k^2)$ pares de nodos a revisar. Total $O(k^2)$, polinomial en el tamaño de la instancia de 3-SAT. ✓

PASO (IV) — EQUIVALENCIA EN AMBAS DIRECCIONES

($\Rightarrow$) Si la fórmula de 3-SAT es satisfacible, entonces G tiene un conjunto independiente de tamaño k. Sea A una asignación que satisface todas las cláusulas. Cada cláusula C_i tiene al menos un literal verdadero bajo A (por definición de “satisfacer”); elegir, para cada cláusula, **uno** de sus literales verdaderos (si hay varios, cualquiera) y tomar el nodo correspondiente. Esto da un conjunto S de exactamente k nodos (uno por cláusula). S es independiente porque: (a) nunca hay dos nodos elegidos de la misma cláusula (se eligió exactamente uno por cláusula, y esos 3 nodos son los únicos conectados por aristas de triángulo entre sí dentro de esa cláusula), y (b) nunca hay dos nodos elegidos que sean literales opuestos de la misma variable (porque ambos fueron elegidos por ser verdaderos bajo la misma asignación A, y una variable no puede ser simultáneamente verdadera y falsa bajo una misma asignación) — así que tampoco hay aristas de conflicto entre los nodos de S.

($\Leftarrow$) Si G tiene un conjunto independiente S de tamaño k, entonces la fórmula de 3-SAT es satisfacible. Como hay exactamente k cláusulas y los nodos de cada cláusula forman un triángulo (a lo sumo 1 nodo de S por cláusula puede estar en un conjunto independiente), y $|S|=k$, S debe contener **exactamente un nodo de cada cláusula** (no puede haber cláusulas con 0 nodos elegidos, porque entonces con k cláusulas y a lo sumo 1 por cláusula, $|S|$ sería $<k$). Construir la asignación A: para cada literal correspondiente a un nodo de S, asignar el valor de verdad que lo hace verdadero (si el nodo es x_j , poner $x_j=1$; si es $\neg x_j$, poner $x_j=0$); a las variables que no aparecen en S, asignarles cualquier valor (por ejemplo 1). Esta asignación está bien definida (no hay contradicciones) precisamente porque S es independiente: si S contuviera simultáneamente un nodo x_j y un nodo $\neg x_j$ (de cláusulas distintas), habría una arista de conflicto entre ellos, contradiciendo que S es independiente. Bajo A, cada cláusula tiene su literal elegido evaluado a verdadero (por construcción de A), así que las k cláusulas quedan satisfechas. ■

VERIFICACIÓN CON UNA INSTANCIA NUMÉRICA CONCRETA

Fórmula 3-SAT con variables x_1, x_2, x_3 y tres cláusulas:

- $C_1 = (x_1 \vee x_2 \vee x_3)$
- $C_2 = (\neg x_1 \vee \neg x_2 \vee x_3)$
- $C_3 = (x_1 \vee \neg x_2 \vee \neg x_3)$

Grafo construido (9 nodos, uno por literal; ver notación (cláusula, posición)):

Nodo	Literal
(0,0)	x_1 (de C_1)
(0,1)	x_2 (de C_1)
(0,2)	x_3 (de C_1)
(1,0)	$\neg x_1$ (de C_2)
(1,1)	$\neg x_2$ (de C_2)
(1,2)	x_3 (de C_2)
(2,0)	x_1 (de C_3)
(2,1)	$\neg x_2$ (de C_3)
(2,2)	$\neg x_3$ (de C_3)

Aristas de triángulo dentro de cada cláusula: $\{(0,0)-(0,1), (0,0)-(0,2), (0,1)-(0,2)\}, \{(1,0)-(1,1), (1,0)-(1,2), (1,1)-(1,2)\}, \{(2,0)-(2,1), (2,0)-(2,2), (2,1)-(2,2)\}$. Aristas de conflicto (misma variable, polaridad opuesta, cláusulas distintas): $(0,0)-(1,0)$ [x_1 vs $\neg x_1$], $(1,0)-(2,0)$ [$\neg x_1$ vs x_1], $(0,1)-(1,1)$ [x_2 vs $\neg x_2$], $(0,1)-(2,1)$ [x_2 vs $\neg x_2$], $(1,1)-(2,1)$ [$\neg x_2$ vs $\neg x_2$ — **no** es conflicto, misma polaridad, no se agrega], $(0,2)-(2,2)$ [x_3 vs $\neg x_3$], $(1,2)-(2,2)$ [x_3 vs $\neg x_3$].

Se verificó por fuerza bruta con un script Python (recorriendo las $2^3=8$ asignaciones posibles de x_1, x_2, x_3 , y todos los subconjuntos de tamaño 3 de los 9 nodos):

Instancia satisfacible → existe Independent Set de tamaño 3.

Fuerza bruta sobre 3-SAT: 5 de las 8 asignaciones posibles satisfacen las 3 cláusulas (ej. $x_1=T, x_2=F, x_3=F$ es una de ellas).

Fuerza bruta sobre el grafo (12 conjuntos independientes de tamaño 3 encontrados sobre $C(9,3)=84$ subconjuntos posibles), por ejemplo $S=\{(0,0), (1,1), (2,0)\} \rightarrow$ literales $\{x_1, \neg x_2, x_1\} \rightarrow$ asignación reconstruida $x_1=True, x_2=False$ (x_3 libre) $\rightarrow$ se verificó por evaluación directa que satisface las 3 cláusulas.

Conclusión numérica: 3-SAT satisfacible = True, Independent Set de tamaño 3 existe = True. Equivalencia confirmada en esta instancia. ✓

Instancia NO satisfacible → NO existe Independent Set de tamaño k.

Se probó además una segunda instancia deliberadamente insatisfacible: $C_1=(x_1 \vee x_1 \vee x_1)$ [fuerza $x_1=1$], $C_2=(\neg x_1 \vee \neg x_1 \vee \neg x_1)$ [fuerza $x_1=0$ — contradicción directa con C_1], $C_3=$

$(x_2 \vee x_3 \vee \neg x_2)$ [cláusula de relleno, siempre satisfacible por sí sola].

Fuerza bruta sobre 3-SAT: 0 de las 8 asignaciones satisfacen las 3 cláusulas a la vez (por la contradicción entre C_1 y C_2).

Fuerza bruta sobre el grafo construido con la misma regla: 0 conjuntos independientes de tamaño 3 (sobre $C(9,3)=84$ subconjuntos revisados).

Conclusión numérica: ambas respuestas dan False, confirmando la equivalencia también en la dirección “no”. ✓ (Script de verificación disponible en el historial de esta sesión; ambas corridas fueron ejecutadas con Python 3.12, recorriendo exhaustivamente asignaciones y subconjuntos — sin atajos heurísticos.)

Esta reducción es literalmente el “gadget” de la sección 8.2 de Kleinberg & Tardos: **1 triángulo por cláusula** (fuerza “a lo sumo un literal por cláusula”) + **1 arista por cada par de literales opuestos** (fuerza “consistencia de la asignación”). Es la plantilla mental a recordar: cualquier reducción hacia Independent Set/Clique/Vertex Cover que involucre SAT probablemente reutiliza esta misma idea de “gadget triangular + aristas de conflicto”.

5. Problemas NP-Completos de referencia

Estos son los problemas “de partida” típicos para reducciones en exámenes de este curso. Ninguno se demuestra desde cero en la práctica del curso (serían demostraciones circulares) — se toman como axiomas de dificultad ya establecidos, y todas las demostraciones nuevas parten de reducir alguno de estos hacia el problema pedido.

Problema	Enunciado (versión decisión)	Reducción típica de origen
SAT	¿Existe asignación que satisface un conjunto de cláusulas (cualquier tamaño)?	Teorema de Cook-Levin (raíz de toda la teoría, no se demuestra en el curso)
3-SAT	Igual que SAT, pero cada cláusula tiene exactamente 3 literales	$\text{SAT} \leq_p \text{3-SAT}$ (se toma como dado en este curso)
Independent Set	¿Existe $S \subseteq V$, $ S \geq k$, sin dos nodos adyacentes?	$\text{3-SAT} \leq_p \text{Independent Set}$ (sección 4)
Vertex Cover	¿Existe $C \subseteq V$, $ C \leq k$, que cubre toda arista (al menos un extremo en C)?	Complemento de Independent Set: C es vertex cover $\Leftrightarrow V \setminus C$ es independent set
Clique	¿Existe $S \subseteq V$, $ S \geq k$, con todas las aristas posibles entre sus nodos?	Complemento de grafo: clique en $G \Leftrightarrow$ independent set en el grafo complemento $\bar{G}$
Set Cover	Dada colección de subconjuntos de U y k , ¿hay k de ellos cuya unión es U ?	$\text{Vertex Cover} \leq_p \text{Set Cover}$
Set Packing	Dada colección de subconjuntos de U y k , ¿hay k de ellos pairwise disjuntos?	$\text{Independent Set} \leq_p \text{Set Packing}$ (universo=aristas, conjunto por vértice=aristas incidentes)
Hamiltonian Cycle	¿Existe un ciclo simple que visita todos los nodos exactamente una vez?	$\text{3-SAT o Vertex Cover} \leq_p \text{Hamiltonian Cycle}$ (gadgets más elaborados)
Subset Sum	Dado un conjunto de enteros y un objetivo B , ¿hay un subconjunto que suma exactamente B ?	$\text{3-SAT} \leq_p \text{Subset Sum}$, o reducciones desde Vertex Cover/Partition

La relación **Independent Set** $\leftrightarrow$ **Vertex Cover** $\leftrightarrow$ **Clique** es la más usada del curso porque es prácticamente gratis una vez que se tiene una de las tres: **C es vertex cover de $G \Leftrightarrow V \setminus C$ es independent set de $G \Leftrightarrow V \setminus C$ es clique en el grafo complemento $\bar{G}$.** Estas tres NO son reducciones “de gadget” — son identidades directas sobre la misma instancia (o su complemento), y conviene tenerlas memorizadas porque destraban ejercicios rápido.

6. Trampas típicas de examen

Trampa 1 — Reducir “al revés”. Para demostrar que X es NP-Hard hay que construir $Y \leq_p X$ (transformar instancias del problema Y conocido difícil, hacia instancias del problema NUEVO). Construir $X \leq_p Y$ en cambio no demuestra nada útil sobre la dificultad de X — ver sección 2.2.

Trampa 2 — Demostrar una sola dirección de la equivalencia. “Instancia sí de Y ” $\Rightarrow$ “instancia sí de X ” NO es suficiente. Hay que demostrar también la recíproca: “instancia sí de X ” $\Rightarrow$ “instancia sí de Y ”. Sin la segunda dirección, la construcción podría estar generando instancias de X que son “sí” por razones que no tienen nada que ver con que la instancia original de Y lo sea (por ejemplo, f podría mapear toda instancia de Y , sea sí o no, a una instancia trivialmente “sí” de X — eso pasaría la primera dirección sin ningún problema, pero es una reducción completamente inválida).

Trampa 3 — Usar una transformación que no es polinomial sin darse cuenta. Ejemplo clásico: una reducción que, dada una instancia de tamaño n , genera un grafo con 2^n nodos “para cubrir todos los casos” — eso arruina la demostración aunque la equivalencia lógica sea perfectamente correcta, porque una reducción exponencial no prueba nada sobre NP-compleitud (cualquier problema decidable se reduce trivialmente a cualquier otro problema no trivial si se permite tiempo exponencial para la transformación). Siempre hay que contar explícitamente cuántos elementos nuevos crea la construcción y en qué tiempo, en función del tamaño de la instancia original — no alcanza con decir “se puede construir”.

Trampa 4 — Confundir NP-Hard con NP-Completo. NP-Hard no exige estar en NP (ver sección 1.4). Si el examen pide demostrar que X es NP-Completo, el Paso (i) ($X \in \text{NP}$, con certificado y verificador explícitos) es **obligatorio** y se suele omitir por apuro al centrarse solo en la reducción — perdiendo puntos aunque la reducción esté perfecta.

Trampa 5 — Elegir mal el problema de partida Y y trabarse. No es un error de rigor, pero cuesta tiempo de examen: si la reducción “no sale” fácil desde el Y elegido, antes de forzarla conviene reconsiderar si otro problema de la lista de referencia (sección 5) tiene una estructura combinatoria más parecida al problema X pedido. Ej.: problemas de “elegir subconjunto con restricciones de compatibilidad de a pares” casi siempre salen más simples desde Independent Set o Set Packing que desde SAT directamente.

Ejercicios de repaso rápido (con respuesta)

#	Enunciado	Respuesta
1	¿Qué significa NP?	"Nondeterministic Polynomial" — verificable en tiempo polinomial (NO "no polinomial")
2	¿ $P \subseteq NP$ es un hecho demostrado o una conjetura?	Demostrado (sección 1.3): todo algoritmo polinomial sirve como su propio verificador
3	¿ $P = NP$ es un hecho demostrado o una conjetura?	Problema abierto — ni demostrado ni refutado
4	¿Un problema NP-Hard tiene que estar en NP?	No — NP-Hard no exige pertenencia a NP; NP-Completo sí
5	Si $A \leq_p B$ y $B \in P$, ¿qué se puede concluir sobre A?	$A \in P$
6	Si $A \leq_p B$ y A es NP-Hard, ¿qué se puede concluir sobre B?	B es NP-Hard (contrapositivo de la fila anterior)
7	Para demostrar que X es NP-Hard, ¿reduzco X a un problema conocido, o un problema conocido a X?	Un problema conocido Y hacia X ($Y \leq_p X$) — nunca al revés
8	¿Alcanza con demostrar una sola dirección de la equivalencia de una reducción?	No, se deben demostrar ambas direcciones ($\Rightarrow$) y ($\Leftarrow$)
9	¿Cuáles son los 4 pasos para demostrar que X es NP-Completo?	(i) $X \in NP$ con certificado+verificador, (ii) elegir Y NP-Completo, (iii) construir $Y \leq_p X$ en tiempo polinomial, (iv) demostrar equivalencia en ambas direcciones
10	Relación entre Vertex Cover e Independent Set sobre el mismo grafo G	C es vertex cover $\Leftrightarrow V \setminus C$ es independent set (identidad directa, no gadget)
11	¿Por qué 3-SAT $\leq_p$ Independent Set usa triángulos por cláusula?	Para forzar que el conjunto independiente elija a lo sumo un literal por cláusula (los 3 nodos de una cláusula están todos conectados entre sí)
12	¿Por qué se agregan aristas entre literales opuestos en la reducción de la sección 4?	Para forzar que el conjunto independiente no pueda elegir simultáneamente x_j y $\neg x_j$ (consistencia de la asignación resultante)

Resumen ultra-rápido (2 minutos)

- **P** = resoluble en tiempo polinomial. **NP** = verificable en tiempo polinomial dado un certificado. **$P \subseteq NP$ siempre** (demostrado); **$P = NP$** sigue abierto.
- **NP-Hard**: todo problema de NP se reduce a él; no necesita estar en NP. **NP-Completo = NP-Hard + está en NP**.
- **$A \leq_p B$** (reducción polinomial): transformación en tiempo polinomial de instancias de A en instancias de B que preserve la respuesta sí/no en ambas direcciones. Consecuencia: si $B \in P$ entonces $A \in P$; contrapositivo: si A es NP-Hard, B también lo es.
- **Para demostrar X NP-Completo**: (i) $X \in NP$ (certificado + verificador polinomial), (ii) elegir Y ya NP-Completo, (iii) construir $Y \leq_p X$ en tiempo polinomial, (iv) demostrar la equivalencia en **ambas** direcciones, explícitamente.
- **Reducir SIEMPRE desde el problema conocido HACIA el problema nuevo** ($Y \leq_p X$), nunca al revés.
- **Ejemplo canónico**: 3-SAT $\leq_p$ Independent Set — 1 triángulo de 3 nodos por cláusula (fuerza “a lo sumo 1 literal por cláusula”) + aristas entre literales opuestos de distintas cláusulas (fuerza consistencia). Verificado numéricamente con una instancia de 3 cláusulas / 3 variables, en ambas direcciones (caso satisfacible y caso insatisfacible).
- **Problemas de referencia**: SAT, 3-SAT, Independent Set, Vertex Cover, Clique (estos tres relacionados por identidad directa sobre complementos), Set Cover, Set Packing, Hamiltonian Cycle, Subset Sum.
- **Trampas de examen**: reducir al revés; probar una sola dirección de la equivalencia; usar una transformación no polinomial sin justificarlo; confundir NP-Hard con NP-Completo (falta demostrar pertenencia a NP); trabarse con un Y mal elegido en vez de reconsiderar la lista de referencia.